

The Full-Stack Challenge: The Collaborative Dynamic Operational Warp

The Scenario

You are building the core execution dashboard for an enterprise logistics team. Multiple coordinators sit on this dashboard simultaneously.

- **The Operation:** They are dynamically grouping urgent, fast-moving shipments into unified dispatch containers.
- **The Chaos:** The weight limits and container costs update via a live network stream every 3 seconds. Multiple coordinators are trying to drag-and-drop the same shipments into different containers at the exact same millisecond.

The Assignment

Develop a localized ReactJS frontend paired with a NodeJS backend that implements a **"Collision-Agile Dynamic Workspace."**

1. Frontend: The Self-Correcting UI Canvas (ReactJS & JavaScript)

- Build a responsive dashboard displaying a list of "Available Shipments" and three "Container Slots." Each container has a strictly locked maximum weight capacity.
- Users must be able to move shipments into containers using an interactive drag-and-drop interface.
- **The AI-Breaker Constraint (Optimistic Latency Inversion):** When a user drops a shipment into a container, the UI must update **instantaneously** (Optimistic UI update) and assume success. However, the NodeJS server takes exactly **2.5 seconds** to process and validate a transaction.
- If the server rejects the transaction (e.g., the container filled up right before the request landed), your React state engine must smoothly **rewind the component state** and physically animate the shipment "snapping back" to its original position. You cannot use a full page reload or block user interactions while waiting for the server.

2. Backend: The Asynchronous State Collision Gate (NodeJS)

- Build a lightweight NodeJS microservice.
- **The Core Data Paradox (The Multi-User Race Condition):** If two browser tabs submit an update to put a shipment into Container A at the exact same time, a classic race condition occurs.
- Write a custom, in-memory state lock mechanism in Node. You **cannot** use external database lock features or third-party Redis locker packages.
- If a collision occurs, the first request is processed. The second request must be dynamically intercepted, calculated as a "State Conflict," and rejected with a specific

payload detailing exactly which millisecond the collision happened, how much fractional capacity was missed, and the logged compute waste.

3. Real-Time Sync & Reconciliation

- Establish a real-time communication pipeline between Node and React.
- If a shipment is successfully locked into a container by User A, User B's screen must instantly grey out or visually remove that specific shipment mid-drag, preventing them from finishing their action smoothly.